

SQL応用講座1

テーブル定義:CREATE文を用いてテーブルを作成します

CREATE TABLE students(
student_no INTEGER PRIMARY KEY
name VARCHAR(256) NOT NULL,
age INTEGER CHECK(age >= 0) DEFAULT 0,
school_no INTEGER,
FOREIGN KEY (school_no)
REFERENCES schools (school_no)
);

テーブル名

データ型が数値(INTEGER)で主キー(PRIMARY KEY)
データ型がマックス255の文字列(VARCHAR)で非ナ
ル(NOT NULL)

列名

age>=0のチェックでデフォルト値が0

school_no列は、schoolsテーブルのschool_noへの外
部キー

制約一覧

PRIMARY KEY	主キー制約
NOT NULL	非ナル(NOT NULL)制約。
UNIQUE	一意性制約。同じ値は禁止する
CHECK	チェック(検査)制約
FOREIGN KEY REFERENCES	参照制約。参照制約のあるデータに紐づくデータも含め強制的に削除するには ON DELETE CASCADE をCREATE文と一緒に記載します

SQL講座(副問い合わせ(サブクエリ))

SQLで取り出した結果を別のSQLで利用すること

SELECTの結果を利用する

**()内での実行結果をFROMで取得する
別名をalbumとする**

```
SELECT
  AVG(album.size)
FROM
(
  SELECT
    SUM(bytes) SIZE
  FROM
    tracks
  GROUP BY
    albumid
) AS album;
```

**# IN句を用いて、
サブクエリで取り出された結果で絞り込む**

```
SELECT
  customerid,
  firstname,
  lastname
FROM customers
# SELECTの結果に該当するレコードで絞り込む
WHERE supportrepid IN (
  SELECT employeeid
  FROM employees
  WHERE country = 'Canada'
);
```

SELECTの一部に含める

```
SELECT albumid,
  title,
  (
    SELECT count(trackid) FROM tracks
    WHERE tracks.AlbumId = albums.AlbumId
  )
  tracks_count
FROM albums
ORDER BY tracks_count DESC;
```

副問い合わせ:existsを利用して存在するものだけ取り出す

SELECT

*

FROM students

WHERE EXISTS(
SELECT * FROM schools where name = '東高校' AND
school_no = students.school_no);

存在するもの
(student_no:1,2,6)だけ取
り出す

列名は何でも良い(1にすることが多い)

studentsのschool_noが1のものだけ取り出される



1	山田太郎	15	1
2	佐藤次郎	16	1
6	佐藤次郎	18	1

SELECT

*

FROM students

WHERE NOT EXISTS(
SELECT * FROM schools where name = '東高校'
AND school_no = students.school_no);

存在しないものだけ取り出す



3	田中三郎	16	2
4	後藤四郎	15	2
5	山田太郎	16	3

studentsテーブル

student_no	name	age	school_no
1	山田太郎	15	1
2	佐藤次郎	16	1
3	田中三郎	16	2
4	後藤四郎	14	2
5	山田太郎	15	3
6	佐藤次郎	18	1

schoolsテーブル

school_no	name
1	東高校
2	西高校
3	北高校

SQL講座(副問い合わせ(サブクエリ))

EXISTSを用いてレコードを絞り込む

CustomersテーブルのCustomerIdが
InvoicesのCustomerIdと合わせて
Invoicesにレコードが存在するものだけを表示

```
SELECT
    *
FROM
    Customers c
WHERE
    EXISTS (
        SELECT
            1
        FROM
            Invoices
        WHERE
            CustomerId = c.CustomerId
    )
```

NOT EXISTSを用いてレコードを絞り込む

CustomersテーブルのCustomerIdが
InvoicesのCustomerIdと合わせて
Invoicesにレコードが存在しないものだけを表示

```
SELECT
    *
FROM
    Customers c
WHERE
    NOT EXISTS (
        SELECT
            1
        FROM
            Invoices
        WHERE
            CustomerId = c.CustomerId
    )
```

SQL講座(JOIN)

テーブルを紐づける一般的な方法

(従来型構文)

紐づいたレコードだけ取り出される

```
SELECT
  st.name, sc.name
FROM
  Students st, Schools sc
WHERE
  st.school_id = sc.id;
```

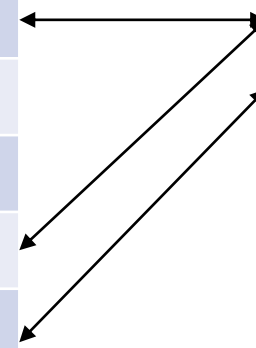
FROMで複数テーブル
を指定し、WHEREで紐
づける

Studentsテーブル

id	name	school_id
1	Taro	1
2	Jiro	3
3	Saburo	5
4	Shiro	1
5	Goro	2

Schoolsテーブル

id	name
1	X高校
2	Y高校



INNER JOIN, JOIN(内部結合)

紐づいたレコードだけ取り出される

```
SELECT
  *
FROM
  Students st
INNER JOIN
  Schools sc
ON st.school_id = sc.id
```

INNER JOINでテーブル
を指定し、ONで条件を
指定する

どちらも同じ結果になるが、
INNER JOINの方が可読性は高く
利用しやすい

INNER JOIN

id	name	school_id	id	name
1	Taro	1	1	X高校
4	Shiro	1	1	X高校
5	Goro	2	2	Y高校

SQL講座(JOIN)

LEFT (OUTER) JOIN(外部結合)

紐づかなかったレコードは
NULLとして取り出される

```
SELECT  
*  
FROM  
  Students st
```

```
LEFT JOIN  
  Schools sc  
ON st.school_id = sc.id
```

LEFT JOINでテーブルを
指定し、ONで条件を指
定する

自己結合(SELF JOIN)

同じテーブル同士で結合する

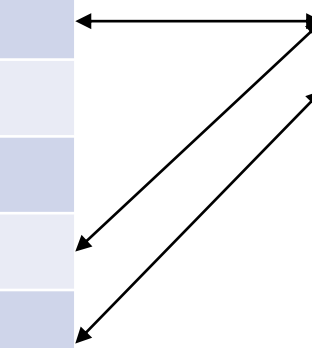
```
SELECT  
  m.firstname || ' ' || m.lastname AS 'Manager',  
  e.firstname || ' ' || e.lastname AS 'Direct report'  
FROM employees e  
INNER JOIN employees m ON m.employeeid = e.managerid  
ORDER BY manager;
```

Studentsテーブル

id	name	school_id
1	Taro	1
2	Jiro	3
3	Saburo	5
4	Shiro	1
5	Goro	2

Schoolsテーブル

id	name
1	X高校
2	Y高校



LEFT JOIN

id	name	school_id	id	name
1	Taro	1	1	X高校
2	Jiro	3	NULL	NULL
3	Saburo	5	NULL	NULL
4	Shiro	1	1	X高校
5	Goro	2	2	Y高校

SQL講座(UNION)

UNION

2つの選択結果から共通要素は1つにして合わせた結果を表示する

```
SELECT v1  
FROM t1  
UNION  
SELECT v2  
FROM t2;
```

ID	名前
1	John
2	Paul
3	George

UNION

ID	名前
3	George
4	Taro
5	Jiro

=

ID	名前
1	John
2	Paul
3	George
4	Taro
5	Jiro

UNION ALL

2つの選択結果を足し合わせた結果を表示する（共通要素は削除されない）

```
SELECT v1  
FROM t1  
UNION ALL  
SELECT v2  
FROM t2;
```

ID	名前
1	John
2	Paul
3	George

UNION ALL

ID	名前
3	George
4	Taro
5	Jiro

=

ID	名前
1	John
2	Paul
3	George
3	George
4	Taro
5	Jiro

ダブっ
て表示

***) 選択結果のカラム数は2つの選択結果間で合わせなければいけない**

SQL講座(UNION)

EXCEPT

1つ目の選択結果から2つ目の選択結果に存在するものを除く

```
SELECT v1  
  FROM t1  
EXCEPT  
SELECT v2  
  FROM t2;
```

ID	名前
1	John
2	Paul
3	George

EXCEPT

ID	名前
3	George
4	Taro
5	Jiro

=

ID	名前
1	John
2	Paul

INTERSECT

2つの選択結果の共通要素だけ取り出す

```
SELECT v1  
  FROM t1  
INTERSECT  
SELECT v2  
  FROM t2;
```

ID	名前
1	John
2	Paul
3	George

INTERSECT

ID	名前
3	George
4	Taro
5	Jiro

=

ID	名前
3	George

***) 選択結果のカラム数は2つの選択結果間で合わせなければならない**

SQL講座(Index)

データを取得する際に、**索引(インデックス)**を利用すると高速でデータを取得できます。

例) 1万件くらいデータのあるstudentsテーブルから、名まえが山田義弘という人を取得する場合、ディスクを全部見るよりもインデックスを見て、山田義弘がディスクのどこにあるのかわかって取得したほうが速い

*) 主キーには初めからインデックスが張られています。他の列にインデックスを張るには手動で作成する必要があります(CREATE INDEX)。よく検索する列（WHEREで絞り込むなど）にはインデックスを張ることも検討しましょう

・インデックスの作成

```
CREATE [UNIQUE] INDEX  
  index_name  
ON table_name(column_list);
```

・インデックスの削除

```
DROP INDEX [IF EXISTS]  
  index_name;
```

豆知識: 何でもかんでもインデックス使うというのは良くない。例えば、山田で始まる人をstudentsテーブルから検索する場合に、studentsテーブルに山田が1%しかいない場合には、インデックスを使う方が速いが、山田が80%存在する場合はインデックスで1つ1つデータを取ってくるより、studentsテーブル内のデータを一括で取ってきてから絞り込む方が速い。

目安として、絞り込む対象が全データ量の15%未満の場合はインデックスを利用したほうが速い(Oracleのマニュアル参照)

チューニングの際に、インデックスを張るかどうかはデータ量と検索する内容次第ということは覚えておきましょう！！

SQL講座(WITH)

テーブルから取得したデータを中間テーブルに格納して、後のSQLで利用する

```
WITH tmp AS(  
  SELECT  
    *  
  FROM users  
  WHERE name='〇〇'  
)
```

usersから絞り込んだ結果をtmpというテーブル名で
中間テーブルに格納する

```
SELECT  
  *  
FROM messages  
WHERE user_id IN(  
  SELECT id FROM tmp  
);
```

中間テーブルtmpを利用する

*) WITH句は副問い合わせでも同じ結果を得ることができ、すべてのRDSで利用できるわけではないです。Mysqlは8.0から